Project Report
BSCS-2nd Semester
Fall 2026.

Subject: Object Oriented Programming

Submitted to:

Sir Mohsin shehzad

Submitted By:

NAME	REG NO
Syed Wasi Haider	25-CS-068

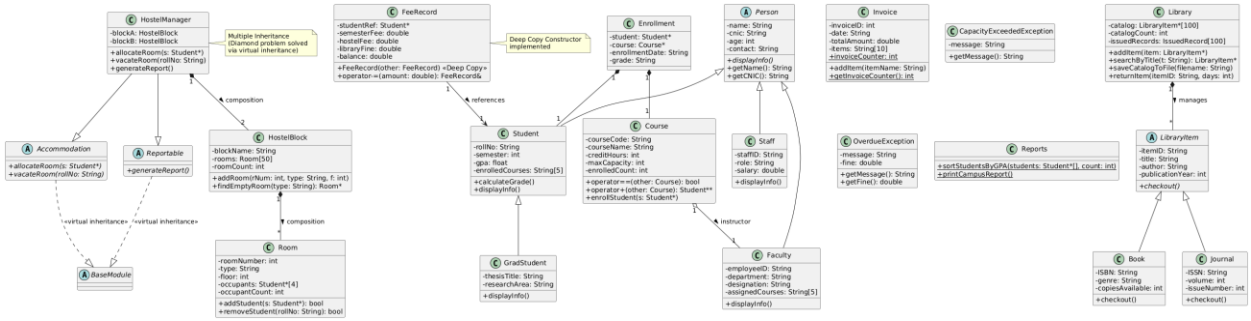
Department of CS
HITEC University, Taxila

Introduction

Managing a university campus manually is a very difficult and time-consuming task. There are many things to handle, like student records, course enrollments, library books, fee payments, and hostel rooms. To solve this problem, we designed and developed the Smart Campus Management System (SCMS) using C++. The main goal of this project was to build a single, integrated software that can handle all these daily operations smoothly. At the same time, this project helped us understand how Object-Oriented Programming (OOP) works in the real world. Instead of just reading about classes and objects in books, we actually built them to solve a real problem. This system is completely console-based, but it is organized in a professional way with separate files for better management.

System Design

The system design is based on a clear UML Class Diagram, which is attached in the docs folder as a PNG file. The core of our design is the Person class, which is an abstract base class. From this base class, we derived the Student, Faculty, and Staff classes using single inheritance. This means all common details like name and CNIC are stored in one place, and specific details are in the child classes. For the library, we used another abstract class called LibraryItem, which is inherited by Book and Journal classes. The hostel management system uses a special design where the HostelManager class inherits from two different interfaces to avoid the diamond problem. We also used composition, which means some classes hold objects of other classes, like the HostelBlock holding an array of Room objects. All these relationships are clearly shown in our UML diagram.



OOP Concepts Implementation

1- Classes & Objects:

We created blueprints for real-world entities. For example, the Person class defines the basic structure.

```
class Person { protected: string name; };
```

2- Encapsulation:

We kept all data members private and provided public getter and setter functions to access them safely.

```
private: string name; public: string getName() { return name; }
```

3- Constructors:

We used default and parameterized constructors to initialize objects properly.

```
Student::Student(string n, string c) : Person(n, c) { rollNo = ""; }
```

4- Destructors:

We used destructors to clean up dynamically allocated memory and prevent memory leaks.

```
Library::~Library() { for(int i=0; i<catalogCount; i++) delete catalog[i]; }
```

5- Single Inheritance:

The Student class inherits directly from the Person class to reuse its properties.

```
class Student : public Person { ... };
```

6- Multi-level Inheritance:

The GradStudent class inherits from Student, which in turn inherits from Person.

```
class GradStudent : public Student { ... };
```

7- Multiple Inheritance:

The HostelManager class inherits from both Accommodation and Reportable classes.

```
class HostelManager : public Accommodation, public Reportable { ... };
```

8- Virtual Inheritance:

To solve the diamond problem in multiple inheritance, we used virtual inheritance for the BaseModule.

```
class Accommodation : virtual public BaseModule { ... };
```

9- Abstract Classes & Pure Virtual:

We created classes that cannot be instantiated and force child classes to implement specific functions.

```
virtual void displayInfo() = 0;
```

10- Runtime Polymorphism:

We used base class pointers to call the overridden functions of child classes at runtime.

```
Person* p = new Student(...); p->displayInfo();
```

11- Operator Overloading:

We overloaded operators to work with custom objects, like comparing two courses.

```
bool Course::operator==(const Course& other) const { return courseCode == other.courseCode; }
```

12- Friend Functions:

We used friend functions to allow external functions to access private members, like printing course details.

```
friend ostream& operator<<(ostream& out, const Course& c);
```

13- Static Members:

We used a static variable in the Invoice class to keep track of the total number of invoices generated.

```
static int invoiceCounter; int Invoice::invoiceCounter = 0;
```

14- Copy Constructor (Deep Copy):

We implemented a deep copy constructor in FeeRecord to ensure dynamic memory is copied correctly.

```
FeeRecord::FeeRecord(const FeeRecord& other) { remarks = new string(*other.remarks); }
```

15- Copy Assignment and Destructor:

We properly handled memory in the Invoice class to avoid double deletion.

```
Invoice::~Invoice() { delete[] items; }
```

16- Search Functions:

We wrote functions to loop through arrays and find specific items, like searching a book by title.

```
if(catalog[i]->getTitle() == t) return catalog[i];
```

17- Array-based Collections:

We used arrays of pointers to store collections of objects, like the library catalog.

```
LibraryItem* catalog[100];
```

18- Arrays of Objects:

We used arrays of objects to manage multiple entities, like occupants in a room.

```
Student* occupants[4];
```

19- Exception Handling:

We created custom exception classes and used try-catch blocks to handle errors like course capacity exceeding.

```
throw CapacityExceededException("Course capacity exceeded!");
```

20- File I/O:

We used fstream to save and load the library catalog data to a text file for persistence.

```
ofstream file(filename); file << title;
```

21- Reporting & Utilities:

We created a separate Reports class with static helper functions to print campus summaries.

```
void Reports::printCampusReport() { cout << "Campus Report"; }
```

22- Memory Management:

We carefully used the new and delete keywords to allocate and free dynamic memory.

```
Book* b1 = new Book(...); delete b1;
```

23- Sorting and Searching:

We used the standard library sort function with a custom comparator to sort students by GPA.

```
std::sort(students, students + count, compareGPA);
```

24- Composition:

We built complex classes by containing objects of other classes, like HostelManager containing HostelBlock.

```
class HostelManager { HostelBlock blockA; };
```

25- Aggregation:

We used pointers to represent "has-a" relationships where the owned object can exist independently, like a Course having a Faculty pointer.

```
class Course { Faculty* instructor; };
```

Module Descriptions

The project is divided into six logical modules. The first module is the Person Hierarchy, which acts as the backbone of the system. It holds all the basic information about anyone on campus and uses inheritance to separate students, faculty, and staff. The second module is Course and Enrollment Management. This module handles course details and allows students to enroll. It also has a waiting list feature and prevents enrollment if the course is full.

The third module is the Library System. It manages books and journals using an array of pointers. It can save and load the catalog from a text file, and it calculates fines if a item is returned late. The fourth module is Fee and Finance Management. It tracks semester fees, hostel fees, and library fines. It generates invoices with auto-incrementing IDs and allows students to make partial payments.

The fifth module is Hostel Management. This was the most complex part, as it uses multiple inheritance and composition. It manages different blocks and rooms, and assigns students to available rooms based on their type. The sixth and final module is Reporting and Utilities. It contains helper functions to format dates, validate inputs, sort students by their GPA, and generate a final consolidated text report of the entire campus status.

GitHub Workflow

We used GitHub for version control to simulate a professional software development environment. We started by creating a new public repository and initializing Git in our local folder. We made sure to add a .gitignore file on the very first day to prevent compiled files like .exe or .o from being uploaded. We followed a strict commit strategy. Instead of uploading everything at the end, we made separate commits for each phase, such as "Add Phase 1 files - Person and Course classes" and "Final submission - all modules complete". This shows a clear history of our progress and proves that the code was written step by step.

Challenges and Solutions

During this project, we faced a few challenges, but we managed to solve them. The first challenge was dealing with memory leaks. Initially, when we copied objects, the program would crash because two objects were trying to delete the same memory. We solved this by implementing a proper Deep Copy Constructor and Copy Assignment Operator, ensuring each object has its own separate memory.

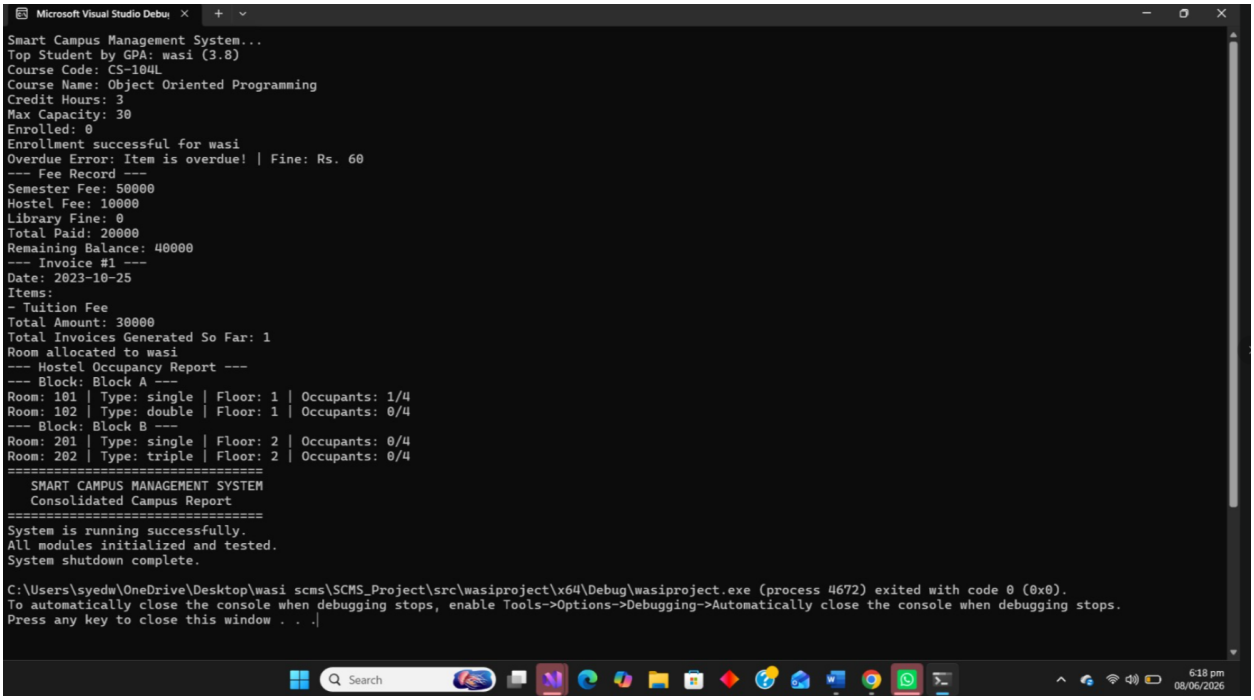
The second challenge was the diamond problem in the Hostel Management module. Since HostelManager needed to inherit from both Accommodation and Reportable, it was getting duplicate copies of the base class. We solved this by using the "virtual" keyword in the inheritance of the base class, which fixed the ambiguity.

The third challenge was reading data back from the text file in the Library module. At first, the program would just read the whole line as a single string. We solved this by using the getline function and parsing the string properly to extract the book details, making the file loading feature actually work.

Testing

We tested every module thoroughly to ensure the system works without errors. We created test objects for students and faculty to verify the inheritance and polymorphism. We tested the course enrollment by trying to add more students than the7. Conclusion

Working on the Smart Campus Management System was a very rewarding experience. Before this project, OOP concepts like virtual inheritance, deep copying, and operator overloading seemed very confusing and theoretical. However, by applying them to a real-world scenario, we finally understood why they are important. We learned how to organize a large codebase into multiple files, how to manage dynamic memory safely, and how to use Git for version control. This project not only improved our C++ programming skills but also taught us how to think like software engineers.



References

I. Deitel, P., & Deitel, H. (2017). C++ How to Program (10th Edition). Pearson.

II. CPPReference.com. (2023). C++ Standard Library Reference. Retrieved from https://en.cppreference.com

III. LearnCPP.com. (2023). Free C++ Tutorials. Retrieved from https://www.learncpp.com